Projet Eco-Sim

Document d’exigences produit (PRD)

Document d'Exigences Produit (PRD) : "Eco-Sim"

Objectif : Créer une plateforme web pour centraliser, analyser et prédire les prix du marché automobile in-game via l'IA.

1. Vision du Produit

Transformer un processus de saisie manuelle fastidieux en un système automatisé capable d'apprendre des données communautaires pour fournir les estimations de prix les plus précises du marché.

2. Modules Fonctionnels

Module A : Ingestion de Données (OCR & IA)

- Upload d'image : L'utilisateur glisse-dépose une capture d'écran de l'interface du véhicule.
- Analyse IA (Vision) :
- Extraire le texte (Marque, Modèle, Réputation, Prix).
- Identifier visuellement la rareté des pièces (par couleur de bordure : Gris, Vert, Bleu, Violet, Jaune).
- Mapper le "Type de voiture" (Eco, Singulière, Rare, Secrète).
- Formulaire de Validation : Présenter les données extraites dans un formulaire éditable pour que l'utilisateur corrige d'éventuelles erreurs avant soumission.

Module B : Base de Données & Moteur de Calcul

- Architecture BDD : Utilisation d'une base de données relationnelle (PostgreSQL) avec trois tables principales :

1. Table Modèles : Marque, Modèle, Type, Prix de base estimé.
2. Table Pièces : Bonus de réputation et bonus de prix par niveau de rareté.
3. Table Observations : Historique de toutes les entrées validées (la source de vérité).

- Algorithme d'Apprentissage :

1. Le système ne doit pas utiliser de constantes "fixes" mais des moyennes pondérées.
2. À chaque nouvelle entrée validée, le système recalcule l'impact moyen d'une pièce "Rare" sur le "Prix Min" pour affiner la formule automatiquement.

Module C : Mode Recherche & Simulateur

- Sélecteurs Dépendants :
3. Menu 1 : Sélection de la Marque.
4. Menu 2 : Sélection du Modèle (filtré selon la marque).
- Configurateur de Pièces : 8 menus déroulants (un par emplacement) permettant de choisir le niveau de rareté (Stock à Légendaire).
- Résultat Dynamique : Affichage en temps réel des estimations (Prix Min, Max, Reco, x2) basé sur les dernières données de la BDD.

3. Exigences Techniques

- Frontend : React ou Vue.js pour une interface fluide.
- Backend : Python (FastAPI ou Flask) – idéal pour manipuler les bibliothèques d'analyse de données.
- Analyse d'image : Utilisation de l'API Vision (Google Cloud Vision ou OpenAI GPT-4o-vision) pour la reconnaissance des pièces par couleur.
- Hébergement : Cloud (Vercel ou Heroku) avec une base de données managée.

4. Expérience Utilisateur (UX)

- Mobile First : L'application doit être parfaitement utilisable sur téléphone pour permettre l'upload direct depuis la galerie photo.
- Feedback Visuel : Un indicateur de "Confiance" sur l'estimation (ex: "Basé sur 150 observations").

Module D : Stratégie de Monétisation

1. Publicité Programmatique (Google AdSense)
- Emplacements stratégiques :
- Bannière "Sticky" : Un encart discret en bas de l'écran sur mobile pour ne pas gêner la navigation.
- Encart de transition : Une bannière horizontale entre le formulaire de validation et l'affichage des résultats de calcul.
- Sidebar (Desktop) : Sur les côtés de l'outil de simulation pour maximiser le taux d'affichage (CPM).
- Politique d'affichage : Les publicités ne doivent jamais masquer les boutons d'upload ou les résultats de calcul critiques.

2. Soutien Communautaire (Don PayPal / Buy Me a Coffee)
- Module de Gratitude : Intégration d'un bouton "Donner" (PayPal) ou un lien "Buy Me a Coffee" bien visible dans le footer (bas de page) et sur la page de résultats.
- Message de soutien : Affichage d'une barre de progression optionnelle ("Aidez-nous à payer l'hébergement des données") pour encourager les utilisateurs réguliers à contribuer.

- Statut "Donateur" : Optionnel — Un système de cookies ou de compte utilisateur simple pour masquer les publicités Google Ads pendant 30 jours pour ceux qui ont fait un don.

Impact sur le développement (Stack Technique) :

- Google AdSense : Nécessite l'intégration du script adsbygoogle.js et une page "Politique de Confidentialité" conforme au RGPD.
- PayPal : Intégration via l'API PayPal SDK ou un simple bouton de lien sécurisé pour minimiser la maintenance.

Schema SQL

Pour une application capable d'apprendre et de s'affiner, le schéma SQL doit séparer les **données brutes** (ce que les utilisateurs envoient) des **coefficients calculés** (ce que l'IA en déduit).
Voici une structure optimisée pour **PostgreSQL** ou **MySQL** :

1. Table car_types

Définit les multiplicateurs de base et les écarts (Min/Max/Reco) par rareté de véhicule (Eco, Rare, Secrète, etc.).

```
CREATE TABLE car_types (
   id SERIAL PRIMARY KEY,
   name VARCHAR(50) UNIQUE, -- ex: 'Singulière', 'Secrète'
   base_ratio DECIMAL(5,2), -- ex: 0.5 pour Singulière
   gap_max_min INT,         -- ex: 142500
   gap_reco_min INT         -- ex: 95000
);
```

2. Table cars

Répertorie les modèles. La base_price est ici la valeur "Stock" de la voiture.

```
CREATE TABLE cars (
   id SERIAL PRIMARY KEY,
   brand VARCHAR(100),
   model VARCHAR(100) UNIQUE,
   type_id INT REFERENCES car_types(id),
   base_reputation INT,
   base_price_min INT -- Le prix min VM quand tout est en Stock
);
```

3. Table part_weights (Le cerveau de l'app)

C'est ici que l'application stocke son "savoir". Ces valeurs évoluent grâce aux moyennes des observations.

```
CREATE TABLE part_weights (
   rarity VARCHAR(50) PRIMARY KEY, -- 'Gris', 'Rare', 'Légendaire'
   bonus_reputation_avg DECIMAL(10,2),
   bonus_price_min_avg DECIMAL(15,2),
```

```
    observation_count INT DEFAULT 0 -- Pour pondérer les nouveaux calculs
);
```

4. Table observations (La source de vérité)

Historique de chaque screenshot validé. Sert à recalculer les moyennes de la table part_weights.

```
CREATE TABLE observations (
    id SERIAL PRIMARY KEY,
    car_id INT REFERENCES cars(id),
    user_id INT, -- Optionnel (si système de comptes)
    rep_total INT,
    price_min_total INT,
    -- Stockage des 8 pièces pour analyse
    engine_rarity VARCHAR(50),
    clutch_rarity VARCHAR(50),
    turbo1_rarity VARCHAR(50),
    turbo2_rarity VARCHAR(50),
    suspension1_rarity VARCHAR(50),
    suspension2_rarity VARCHAR(50),
    transmission_rarity VARCHAR(50),
    tires_rarity VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Comment l'automatisation fonctionne (Logique de calcul) :

Pour que l'application "s'améliore", on utilise un **Trigger** ou une **Tâche de fond** :

1. **Nouvelle observation :** Un utilisateur valide une *Takada Kame* avec un *Moteur Rare*.
2. **Soustraction :** L'app prend le price_min_total et soustrait le base_price_min de la voiture.
3. **Mise à jour :** Elle met à jour la table part_weights en faisant une moyenne :
   - *Nouveau Bonus Moyen = ((Ancien Bonus * Nb Obs) + Nouveau Bonus) / (Nb Obs + 1)*
4.

Avantages de ce schéma :

- **Évolutif :** Si les développeurs du jeu changent les prix, l'application s'ajustera toute seule après quelques nouveaux screenshots.
- **Recherche rapide :** Les menus déroulants Marque > Modèle sont ultra-rapides car indexés.
- **Monétisation :** Tu peux facilement ajouter une table users pour gérer les dons ou masquer les pubs.

Dictionnaire de Données Initiales

Voici le dictionnaire de données initial, structuré pour peupler votre base de données SQL. Ce tableau intègre les constantes identifiées pour les types Singulière et Secrète, ainsi que les bonus de pièces qui serviront de base à l'apprentissage du système.

1. Initialisation des Types de Véhicules (car_types)

Ces valeurs fixent les règles de calcul de base (écarts de prix) pour chaque catégorie de rareté.

Type	base_ratio	gap_max_min	gap_reco_min
Eco / Rare	0.40	2 450 000	700 000
Singulière	0.50	142 500	95 000
Légendaire	1.00	Varie (ex: 91M)	Varie (ex: 14M)
Secrète	~0.95	237 500 000	25 000 000

2. Initialisation des Poids des Pièces (part_weights)

C'est le "cerveau" de l'application. Ces bonus s'ajoutent à la valeur de base du véhicule en fonction des pièces détectées sur le screenshot.

Rareté (Pièce)	Bonus Réputation (Moyenne)	Bonus Prix Min (Moyenne)
Stock	0	0
Gris	+1 240	+3 612
Singulière	+1 970	+10 815
Rare	+3 120	+54 040
Épique	+4 960	+324 162
Légendaire	+10 925	+3 500 000*

*Valeur estimée d'après la Barnett Spirit, à affiner par l'application.

3. Exemples de Modèles de Référence (cars)

Ces modèles serviront de "points d'ancrage" pour valider les premières analyses OCR.

Marque	Modèle	Type	Base Price Min (Stock)	Base Reputation
Takada	Kame	Singulière	47 500	475
Bomber	E30	Rare	350 000	2 800
Barnett	Spirit	Secrète	18 750*	18 750

Ferrini	Salerno	Secrète	22 100*	578

*Valeurs basées sur vos dernières observations PDF.

Note technique pour le développement :
Pour le calcul du Prix x2, le système devra utiliser la formule dynamique suivante basée sur la table part_weights :
Prix_x2 = Prix_Min + (Reputation_Totale * Multiplicateur_K)
Le multiplicateur K semble osciller entre 2.2 (bas niveau) et 2.4 (haut niveau).

Requete d’insertion SQL

C'est noté, l'erreur est rectifiée. Dans l'univers de ce jeu, le modèle **578** de la marque **Ferrini** est effectivement un véhicule de catégorie **Secrète** avec une réputation de base très élevée (**22 100**), ce qui le place dans le haut du panier avec les modèles Barnett Spirit.
Voici le dictionnaire de données corrigé pour cette entrée spécifique et prêt pour l'intégration SQL :

Correction du Modèle : Ferrini 578

Propriété	Donnée Rectifiée	Note
Marque	Ferrini	
Modèle	578	
Type	Secrète	
Réputation (Stock)	22 100	Réputation de base sans pièces.
Prix Stock (Min VM)	*À définir*	Sera calculé par l'app lors du premier screenshot "Stock".

Requête SQL d'insertion (Données de Référence)

Voici le script pour injecter ces bases solides dans tes tables. Il inclut la correction de la Ferrini et les paliers de pièces.

```
-- 1. Insertion des Types
INSERT INTO car_types (name, base_ratio, gap_max_min, gap_reco_min) VALUES
('Singulière', 0.50, 142500, 95000),
('Rare', 0.40, 2450000, 700000),
('Secrète', 0.95, 237500000, 25000000);

-- 2. Insertion des Modèles de base
INSERT INTO cars (brand, model, type_id, base_reputation) VALUES
('Takada', 'Kame', 1, 475),
('Bomber', 'E30', 2, 2800),
('Ferrini', '578', 3, 22100),
```

('Barnett', 'Spirit', 3, 18750);

-- 3. Insertion des Poids des Pièces (Cerveau initial)
INSERT INTO part_weights (rarity, bonus_reputation_avg, bonus_price_min_avg)
VALUES
('Gris', 1240.00, 3612.00),
('Singulière', 1970.00, 10815.00),
('Rare', 3120.00, 54040.00),
('Epique', 4960.00, 324162.00),
('Légendaire', 10925.00, 3500000.00);

Algorithme d’apprentissage

Pour que ton application ne soit pas juste un calculateur figé, mais un système qui s'affine à chaque capture d'écran, l'algorithme doit fonctionner par **rétro-ingénierie**.
Voici la logique de l'algorithme d'apprentissage (ou *Learning Engine*) que tes développeurs devront implémenter.

1. Le Principe : La Soustraction des Connus

L'idée est simple : pour apprendre la valeur d'une pièce inconnue (ou affiner une connue), l'algorithme "nettoie" le prix total en soustrayant tout ce qu'il connaît déjà avec certitude.
Formule de base :
$Valeur_Extraite = Prix_Total_Observé - (Base_Véhicule + \sum Valeurs_Autres_Pièces_Connues)$

2. L'Algorithme en 3 Étapes

Étape 1 : Isolation de la donnée (Extraction)

À chaque nouvelle observation validée par l'utilisateur, le système calcule l'écart pour chaque pièce i présente sur le véhicule :

1. Il récupère le base_price_min du modèle dans la table cars.
2. Il identifie les pièces $n-1$ dont il connaît déjà le poids.
3. Il isole la valeur résiduelle pour la pièce cible.

Étape 2 : Pondération (Moyenne Mobile)

Le système ne remplace pas l'ancienne valeur par la nouvelle, il l'ajuste. On utilise le observation_count pour donner moins de poids à une erreur isolée.
L'équation de mise à jour dans part_weights :
$$Nouveau_Bonus = \frac{(Ancien_Bonus \times Nb_Obs) + Valeur_Extraite}{Nb_Obs + 1}$$

Étape 3 : Auto-Correction du Modèle (Découverte)

Si le système détecte une nouvelle voiture (ex: une marque jamais vue), il crée une entrée temporaire :

- Si l'utilisateur entre une voiture avec des pièces, l'app fait le calcul inverse : $Prix_Total - \sum Pièces = Base_Véhicule_Inconnue$.
- Elle enregistre cette base_price_min dans la table cars.

3. Exemple Concret d'Apprentissage

Imaginons que le bonus **Légendaire** soit actuellement estimé à **3 500 000** (basé sur 1 observation).

1. Un utilisateur envoie un screenshot d'une *Ferrini 578* avec **1 pièce Légendaire**.
2. Le Prix_Min observé est de **27 038 922**.
3. L'algorithme sait (via le PDF) que la base de cette Ferrini est d'environ **23 500 000**.
4. **Calcul :** $27\,038\,922 - 23\,500\,000 = 3\,538\,922$ (C'est la nouvelle valeur extraite).
5. **Mise à jour :** $\frac{(3\,500\,000 \times 1) + 3\,538\,922}{2} = \mathbf{3\,519\,461}$.
6. La nouvelle référence pour le "Légendaire" devient **3 519 461**.

4. Intégration dans le PRD (Exigence Data)

- **Seuil de confiance :** Une valeur n'est considérée comme "Stable" qu'après **10 observations** cohérentes (écart-type faible).
- **Alerte d'Anomalie :** Si une observation s'écarte de plus de 20% de la moyenne actuelle, l'application doit la marquer pour "Validation Manuelle" par l'administrateur afin d'éviter qu'un faux screenshot ne casse l'algorithme.

Souhaites-tu que je prépare un exemple de code (en Python) qui illustre cette fonction de mise à jour pour ton équipe technique ?

Exemple de code:

Voici un exemple de script **Python** utilisant une logique de classe. Ce code illustre comment le "cerveau" de l'application traite une nouvelle observation pour affiner ses connaissances.
Il utilise la méthode de la **moyenne pondérée cumulative**, ce qui permet au modèle d'être de plus en plus stable à mesure que les données s'accumulent.

```
class EcoSimLearningEngine:
    def __init__(self, db_part_weights):
```

```
        # On simule la table 'part_weights'
        self.part_weights = db_part_weights

    def learn_from_observation(self, observation):
        """
        Analyse une observation et met à jour les moyennes des bonus de pièces.
        observation = {
            'base_car_price': 47500,
            'total_price_min': 58315,
            'parts': ['Singulière', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock']
        }
        """
        # 1. Calculer le bonus total apporté par les pièces
        total_bonus_observed = observation['total_price_min'] -
observation['base_car_price']

        # 2. Identifier les pièces non-Stock (celles qui portent le bonus)
        active_parts = [p for p in observation['parts'] if p != 'Stock']

        if not active_parts:
            return "Aucune pièce active à analyser."

        # 3. Répartir le bonus (Hypothèse d'additivité simple confirmée par nos tests)
        # Note : Si plusieurs pièces différentes, on isole la cible.
        # Ici, on simplifie : on divise le bonus par le nombre de pièces actives.
        individual_contribution = total_bonus_observed / len(active_parts)

        # 4. Mettre à jour la base de données (Moyenne pondérée)
        for rarity in set(active_parts):
            old_avg = self.part_weights[rarity]['avg_price']
            count = self.part_weights[rarity]['count']

            # Formule de la moyenne mobile
            new_avg = ((old_avg * count) + individual_contribution) / (count + 1)

            # Mise à jour des valeurs
            self.part_weights[rarity]['avg_price'] = round(new_avg, 2)
            self.part_weights[rarity]['count'] += 1

        return f"Mise à jour effectuée. Nouveau bonus pour {active_parts[0]}:
{self.part_weights[active_parts[0]]['avg_price']}"

# --- EXEMPLE D'UTILISATION ---

# Données initiales (notre dictionnaire)
db_weights = {
    'Singulière': {'avg_price': 10800, 'count': 10},
    'Epique': {'avg_price': 324000, 'count': 5}
}
```

```
engine = EcoSimLearningEngine(db_weights)

# Nouvelle observation : Une voiture avec 1 pièce Singulière
# Le prix observé suggère un bonus de 10 815
nouvelle_entree = {
    'base_car_price': 47500,
    'total_price_min': 58315, # 47500 + 10815
    'parts': ['Singulière', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock', 'Stock']
}

print(engine.learn_from_observation(nouvelle_entree))
# Résultat : Le bonus "Singulière" va passer de 10800 à 10801.36
```

Pourquoi ce code est efficace pour ton projet :

1. **Stabilité :** Le count (nombre d'observations) agit comme un frein. Plus tu as de données, moins une seule erreur de lecture IA (OCR) ne pourra fausser la moyenne globale.
2. **Transparence :** Les développeurs peuvent facilement ajouter une vérification d'écart-type. Si individual_contributionest trop éloigné de old_avg, le script rejette l'entrée et demande une validation humaine.
3. **Évolutivité :** On peut facilement adapter le script pour qu'il apprenne aussi les bonus de **Réputation** de la même manière.

UX/UI

Module E : Expérience Utilisateur (UX) & Interface (UI)

1. Le "Dashboard" (Écran d'accueil)

L'écran est divisé en deux zones d'action immédiate :

- Zone Haute (Simulateur) : "Calculer un prix" (Accès direct aux menus déroulants Marque/Modèle).
- Zone Basse (Contribution) : "Analyser un Screenshot" (Gros bouton d'upload avec icône appareil photo).

2. Le Flow de Validation (Après l'Upload)

Une fois l'image analysée par l'algorithme Vision, l'application ne doit jamais enregistrer les données sans confirmation. Elle affiche un formulaire de récapitulatif :

- Aperçu de l'image : Une version miniature de la photo avec des cadres colorés sur les zones lues par l'IA (pour rassurer l'utilisateur).
- Champs pré-remplis :
- Marque/Modèle (Détectés).
- Réputation (Détectée).

- Grille des 8 pièces : Une grille de 8 icônes colorées selon la rareté détectée. L'utilisateur peut cliquer sur une icône pour corriger manuellement la rareté (ex: passer de "Gris" à "Singulier" si l'IA a fait une erreur).
- Bouton "Valider & Contribuer" : Envoie les données vers l'algorithme d'apprentissage.

3. Le Module de Résultat (La "Fiche de Prix")

Une fois la recherche ou l'upload terminé, les résultats sont présentés sous forme de cartes comparatives :

Type de Prix	Montant Estimé	Indice de Fiabilité
Prix Minimum	22 949 760	🟢 Haute (150+ obs)
Prix Recommandé	23 649 760	🟢 Haute
Prix Maximum	25 399 760	🟡 Moyenne
Prix x2	45 051 562	🔴 Basse (3 obs)

UX Trick : Ajouter un bouton "Copier" à côté de chaque prix pour faciliter le collage dans le chat du jeu ou sur un forum.

4. Gestion des pièces "Stock" (Logique UI)

Dans le configurateur manuel (mode recherche), les pièces "Stock" doivent être le choix par défaut.

- Visualisation : Une icône "Vide" ou un cadre grisé.
- Impact Algorithmique : L'UI doit envoyer une valeur 0 ou null au moteur de calcul pour ces emplacements afin qu'aucun bonus ne soit ajouté à la base du véhicule.

5. Intégration de la Monétisation (UI)

- Ads : Un bandeau publicitaire fixe en bas de l'écran (Bottom Anchor Ad).
- Don : Un petit bouton flottant avec une icône de café (Buy Me a Coffee) en bas à droite de l'écran de résultats.

Outils à utiliser

Pour lancer une application qui traite des images et de l'IA sans se ruiner, l'objectif est d'utiliser le modèle "Free Tier" (niveaux gratuits) des services Cloud.
Voici la stack "Budget-Friendly" (économique) recommandée pour Eco-Sim :

1. Hébergement et Backend (Gratuit pour commencer)

- Vercel ou Render : Idéal pour héberger le Frontend (React/Vue) et le Backend (Python/FastAPI).
- Coût : Gratuit pour les projets personnels avec un trafic modéré.
- Supabase (Alternative à PostgreSQL) : Pour ta base de données SQL.
- Coût : Gratuit jusqu'à 500 MB de données (ce qui représente des milliers d'observations de voitures). Inclut aussi la gestion des utilisateurs si besoin.

2. Analyse d'Image / OCR (Le poste le plus cher)

C'est ici que l'IA intervient. Pour limiter les frais :

- Tesseract.js : Une bibliothèque OCR open-source qui tourne directement dans le navigateur de l'utilisateur.
- Coût : 0 € (utilise la puissance du téléphone de l'utilisateur).
- Limitation : Moins précis que Google Cloud Vision pour les couleurs.
- Google Cloud Vision API : Pour une détection parfaite des couleurs de bordure des pièces.
- Coût : Gratuit pour les 1 000 premières analyses par mois. Ensuite, environ 1,50 € par tranche de 1 000 images.

3. Frontend et Design (Gratuit)

- Tailwind CSS : Pour créer une interface mobile moderne et rapide sans designer.
- Lucide React : Pour des icônes de voitures et de moteurs gratuites et élégantes.
- Figma : Pour dessiner ton interface avant de coder (Gratuit pour 3 projets).

4. Monétisation et Dons (Frais uniquement sur les ventes)

- Google AdSense : Inscription gratuite. Tu gagnes de l'argent dès que les gens voient les pubs.
- Buy Me a Coffee : Très simple à intégrer. Ils prennent une petite commission (5%) uniquement si tu reçois un don.

Résumé du budget de lancement :

Poste	Outil	Coût mensuel

Serveur & BDD	Supabase + Render	€ 0
Analyse Image	Tesseract (JS)	€ 0
Nom de domaine	Namecheap / OVH	~10 € / an
TOTAL		< 1 € / mois

Conseil d'ami pour le développement :

Commence par utiliser Tesseract.js pour lire le texte (Marque, Modèle, Réputation). Pour la couleur des pièces, demande aux utilisateurs de cliquer sur les icônes pour confirmer la couleur lors de la phase de lancement ("Bêta"). Cela te permettra de rester à 0 € de frais d'IA le temps que l'application devienne populaire.
Veux-tu que je prépare un petit guide sur la structure des dossiers pour tes développeurs (l'arborescence du projet) ?

ARBORESCENCE RECOMMANDEE:
Cette structure sépare bien le Frontend (ce que l'utilisateur voit), le Backend (le cerveau) et la Logique IA.

📂 Structure du projet : Eco-Sim

```
eco-sim/
├── backend/              # API Python (FastAPI / Flask)
│   ├── app/
│   │   ├── main.py        # Point d'entrée de l'API
│   │   ├── database.py    # Connexion Supabase / PostgreSQL
│   │   ├── models.py      # Définition des tables SQL (SQLAlchemy)
│   │   ├── schemas.py     # Validation des données (Pydantic)
│   │   ├── learning_engine.py # L'algorithme d'apprentissage (notre script)
│   │   └── routes/        # Les différents points d'accès
│   │       ├── analyze.py  # Route pour l'upload d'image
│   │       ├── simulate.py # Route pour le calcul des prix
│   │       └── admin.py    # Route pour la validation des données
│   ├── requirements.txt    # Dépendances (FastAPI, SQLAlchemy, etc.)
│   └── .env                # Variables secrètes (Clés API, URL BDD)
│
├── frontend/             # Interface React / Vite.js
│   ├── src/
│   │   ├── components/    # Composants réutilisables (Boutons, Grilles)
```

```
|   |   ├── hooks/        # Logique de récupération des données
|   |   ├── pages/        # Dashboard, Validateur, Recherche
|   |   ├── utils/        # Formateurs de prix (ex: 22M au lieu de 22000000)
|   |   └── App.jsx       # Navigation principale
|   ├── public/           # Images, icônes et favicon
|   └── tailwind.config.js  # Configuration du style
|
├── ai_service/           # Module optionnel (si OCR déporté)
|   └── processor.py      # Logique Tesseract ou Google Vision
|
└── docs/                 # Ton PRD et Schéma SQL
```

🛠️ **Points clés pour les développeurs**

1. Communication API (backend/routes)

- Le frontend envoie une image en POST à /analyze.
- Le backend renvoie un JSON contenant les détections.
- Une fois validé, le frontend envoie un POST à /confirm pour déclencher le learning_engine.py.

2. Variables d'environnement (.env)

Il est crucial de ne jamais écrire les clés API (Google Vision, PayPal, Supabase) directement dans le code. Elles doivent être stockées dans le fichier .env pour la sécurité.

3. Gestion de l'état (Frontend)

L'utilisation de **React Query** ou **SWR** est recommandée pour gérer la mise à jour des menus déroulants (Marque > Modèle) sans recharger la page.